

Final Project

AI Safety Course

University of Tübingen · Summer Semester 2026

The final project is 50% of the final grade. It runs for roughly four weeks, from **Monday 22 June** to the **final presentations on 20 July**, and you'll work in **teams of three** (by default the same teams as before, but changes are possible). The point is to do a real, if small, piece of safety research: pick a question you are actually interested in, work on it for the project, and tell us what you found. You will work closely with our TAs to shape the scope of the project and consult about implementation details. Please use Slack actively in case you need to clarify something!

You're encouraged to use LLM agents as coding assistants here for the low-level implementation details so you can spend your time on the ideas and the experiments.

Submission form: <https://forms.gle/DTuDi6fBRXLHwjAa7> (deadline: July 20 at 10am).

Final presentations: July 20 from 10am to 2pm (using our exercise session and lecture slots).

Two kinds of project

There are two general types of projects you can pursue.

1. **Extending an existing paper.** Take a recent result and rebuild the central claim yourself, for example the main figure or table from the paper. Then contribute an extension that tells us something new: does the effect generalize to a different model family? A different dataset? Does it remain when you change the phrasing of the prompt, the scale of the experiments, or other assumptions the authors relied on? A good way to think about this is as stress-testing. A lot of safety results have the flavour of “we tried a thing and it worked,” and figuring out *when it stops working* is a useful research contribution. The expectation here is that you go past re-running someone's repository; there should be a question of your own added.
2. **Pursuing a new research idea.** If you'd rather start from a new question, feel free to do that. Build a small benchmark for a failure mode you think is under-measured, design an evaluation, probe something you have not seen pinned down yet. These are riskier, so we will be more lenient in grading: we care less about absolute numbers and more about whether you had a clear reason behind the question, whether your evaluation actually measures what you claim, and whether you did the reading to know where how your work sits relates to what already exists.

Coming up with an idea

The two sketches below are only there to calibrate scope and to show the *kind* of addition that makes a project more than a simple re-run. If you want a sense of the level we're after, look at the [Harvard](#)

[projects from last year](#). One team measured the data-poisoning rate threshold up to which an LLM backdoor was still effective; another showed that the chain-of-thought does not faithfully represent the output once you put two models in a competitive game. The most important advice is to have a clear question and an experimental evaluation that supports it. Here are two other example ideas:

Example 1. Prompt-injection defenses are usually reported with a single attack-success number on a single benchmark. Take a tool-using agent in a setting like AgentDojo, reproduce the main result for one published defense (spotlighting, a detector, delimiting, whatever you like) and the same or different model than the ones covered in the paper, and then test whether the defense is overfit to the fixed set of attack strings the benchmark has. Paraphrase the injected instructions, write a small held-out set of attacks the defense wasn't tuned against, or write adaptive attacks (attacks assuming the attacker knows exactly which defense is in place and which create injections aimed at evading that specific mechanism). Report the gap between the non-adaptive and adaptive robustness. This connects to the agent-security lecture.

Example 2. Steganographic collusion between agents. Set up a small three-model scenario: two agents share a goal that's easier to reach if they coordinate in a way a third model, the monitor, is supposed to catch and block. The question is whether the two can pass hidden information past the monitor through otherwise-innocent messages, and the limits of their success. Does inserting a paraphrasing step between the agents destroy the hidden signal? Does letting the monitor read their chain-of-thought help? You'd be building a tiny testbed and a clear metric for "information got through undetected," which ties to the multi-agent-safety lecture on collusion and steganography.

What to submit, and when

As soon as your team has a direction, send us your **project direction plus a one-page description** via the following link: <https://forms.gle/izJDRp1GraeGyUqe7>. Sooner is better: once you submit, we match you with a TA who'll give you guidance. This step is optional — you can run the whole project without a TA if you want to — but the teams who submit early get the most out of having someone to talk to. Leaving it to the last week means the TA can't considerably help you, especially in scoping and directions.

The one page should cover:

- your team details;
- which shape of project you're doing;
- the paper and result you're reproducing, or the question you're asking;
- roughly how you plan to go about it;
- how you'll evaluate whether it worked.

Timeline

We don't grade intermediate steps of the project. However, we provide an example of what your progress could look like:

When	What
From Mon 22 June	Settle on a direction, submit the one-pager as soon as it's ready, get matched with a TA.
Week 1 (22–28 June)	Scope it down, read relevant papers, get a baseline running.
Week 2 (29 June – 5 July)	Reproduce the main result properly, or build out the benchmark / experiment.
Week 3 (6–12 July)	The core of the project: the extra conditions and the analysis.
Week 4 (13–19 July)	Start consolidating the experiments, writing up your findings, making legible figures, designing the slides.
Monday 20 July (10am)	Final presentations. Project due.

Final Submission Materials

We will grade projects based on:

- A **writeup** up to **six pages** using the [standard ICML format](#) in a concise way. References and appendix are not counted towards the page limit.
- A **final presentation** on 20 July, we will have about ten minutes per team plus questions. Walk us through the question, what you did, what you found, and what you'd do next. We'll confirm exact timing closer to the date once we know how many teams there are.
- Your **code**, in a repository, with enough of a README that someone could rerun your main results.

Grading Rubric

Out of 100:

Component	Pts	What a strong version looks like
Framing & related work	15	You know the prior work that matters, you situate your project against it, and you can say in a sentence why the question is worth asking.
Technical approach	15	The method is sound and the work was non-trivial. You reproduced the real result, or built a real evaluation. In both cases your work went beyond wrapping an existing repo.

Component	Pts	What a strong version looks like
Results & evaluation	15	The experiments support what you claim. Your evaluations are principled and measure the effect you wanted to. You have good controls and baselines for your experiments.
Clarity & uncertainty	10	The writing is clear, the figures are readable, and the limitations section is well thought out. If you are unsure about some effect, conclusion or implication, your writing accurately conveys this uncertainty.
Code & reproducibility	20	The repository runs and someone else could easily reproduce your main result.
Final presentation	25	You explain the work clearly in the time you have, and you handle questions thoughtfully.
Total	100	

Practical notes

Use the compute options that were available during the mini-projects (free Colab, the Gottingen cluster, and Modal). If you're unsure how much compute your idea will require, that's a good thing to raise with your TA when you're matched.

If anything here is unclear, always feel free to ask on Slack.

This project setup borrows heavily, with thanks, from [Harvard's first AI safety course](#) (CS 2881r, taught by Boaz Barak), and from Roy Rinberg's [public reflections on TA-ing it](#).